

Natural Language Processing 2

How do RNNs Help NLP?

Fatemeh Dehrouyeh



FANSHAWE

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)
 - Text Generation
 - Translation

Current Section

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)

The True Nature of RNNs

RNNs are sequence models.

They are designed to process data step by step, where the order of the input matters.

Main idea:

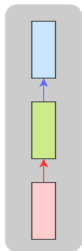
- RNNs read one token at a time.
- They keep a hidden state as memory.
- The hidden state carries information from previous tokens.
- This makes RNNs useful for ordered data.

Common use cases:

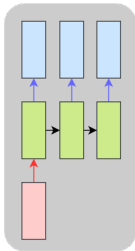
- Text classification, such as sentiment analysis
- Text generation, such as next word prediction
- Sequence labeling, such as named entity recognition
- Time series forecasting

Input-Output Structures

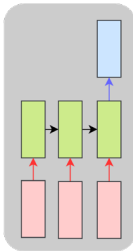
one to one



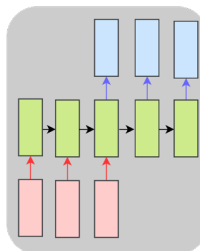
one to many



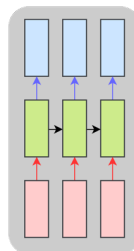
many to one



many to many



many to many



Input-Output Structures in Deep Learning

Different models can be described by how many inputs and outputs they process.

- **One-to-One:** one input gives one output.

image $\rightarrow$ class label

Example: image classification.

- **One-to-Many:** one input generates a sequence.

image $\rightarrow$ caption words

Example: image captioning.

- **Many-to-One:** a sequence gives one output.

review words $\rightarrow$ sentiment label

Example: sentiment classification.

- **Many-to-Many:** a sequence gives a sequence.

previous tokens $\rightarrow$ next tokens

Example: text generation.

English sentence $\rightarrow$ French sentence

Example: machine translation.

Our Focus in RNN Applications

Our focus is on two important RNN structures:

Many-to-One

Sentiment analysis, where many tokens produce one label.

Many-to-Many

Text generation and translation, where a sequence produces another sequence.

Other Input-Output Structures

The other two structures are usually handled differently:

One-to-One

Image classification, where one image produces one label.
This is usually handled with CNNs or Vision Transformers.

One-to-Many

Image captioning, where one image produces a sequence of words.
This is usually handled with a CNN encoder and an RNN/LSTM decoder.

Current Subsection

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)

How RNNs Are Used for Classification

RNNs are used for classification by reading each document as a sequence of tokens.

Important idea:

- Each document may have a different number of tokens.
- Neural networks need inputs with the same shape.
- Therefore, we choose a fixed sequence length.
- Short documents are padded.
- Long documents are truncated.

For example, if we choose:

$$\text{sequence length} = 50$$

then every document must contain exactly 50 token positions.

Then the RNN reads the document token by token:

$$\text{token}_1 \rightarrow \text{token}_2 \rightarrow \text{token}_3 \rightarrow \cdots \rightarrow \text{token}_{50}$$

RNN Input Shape for Classification

RNN reads each token embedding vector one by one.

X shape = (number of documents, sequence length, embedding dimension)

Example:

(1000, 50, 100)

This means:

- 1000 documents
- each document has 50 token positions
- each token is represented by a 100-dimensional embedding vector
- the RNN processes the 50 tokens in order
- word order is preserved

Important: Sequence length is very similar to the lookback window L in time series.

How the RNN Produces the Class Label

After reading all tokens, the RNN creates a final hidden state.

This final hidden state summarizes the document based on word order and context.

Then we pass it to a Dense layer for classification.

word embeddings $\rightarrow$ RNN $\rightarrow$ final hidden state $\rightarrow$ Dense layer $\rightarrow$ label

Example:

“The movie was not good”

The RNN reads:

The $\rightarrow$ movie $\rightarrow$ was $\rightarrow$ not $\rightarrow$ good

Then it predicts a label:

Negative

Just to Remember: How FFNNs Work for Classification

Feedforward neural networks do not read text token by token.

They receive one fixed-size vector for each document.

There are two common ways to create this vector:

- TF-IDF vector
- Averaged word embedding vector

X shape = (number of documents, number of features)

Example:

(1000, 100)

This means:

- 1000 documents
- Each document is represented by one 100-dimensional vector
- The 100 features can be TF-IDF features or embedding dimensions
- Word order is lost

RNN vs FFNN

**RNNs read text as an ordered sequence of tokens,
while FFNNs receive one fixed-size vector for the whole
document.**

Code: IMDB Sentiment Analysis Using RNNs

Dataset Link:

IMDB Movie Reviews dataset

	review	sentiment
1	One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. The	positive
2	A wonderful little production. The filming technique is very unassuming- very old-time-BB	positive
3	I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air con	positive
4	Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his parents are	negative
5	Petter Mattei's "Love in the Time of Money" is a visually stunning film to watch. Mr. Mattei offers us a	positive
6	Probably my all-time favorite movie, a story of selflessness, sacrifice and dedication to a noble cause, I	positive
7	I sure would like to see a resurrection of a up dated Seahunt series with the tech they have today it w	positive
8	This show was an amazing, fresh & innovative idea in the 70's when it first aired. The first 7 or 8 years	negative
9	Encouraged by the positive comments about this film on here I was looking forward to watching this f	negative
10	If you like original gut wrenching laughter you will like this movie. If you are young or old then you will	positive
11	Phil the Alien is one of those quirky films where the humour is based around the oddness of everythin	negative
12	I saw this movie when I was about 12 when it came out. I recall the scariest scene was the big bird eat	negative
13	So im not a big fan of Boll's work but then again not many are. I enjoyed his movie Postal (maybe im t	negative
14	The cast played Shakespeare. Shakespeare lost. I appreciate that this is trying t	negative
15	This a fantastic movie of three prisoners who become famous. One of the actors is george clooney and	positive
16	Kind of drawn in by the erotic scenes, only to realize this was one of the most amateurish and unbelie	negative
17	Some films just simply should not be remade. This is one of them. In and of itself it is not a bad film. B	positive
18	This movie made it into one of my top 10 most awful movies. Horrible. There wasn't a cor	negative
19	I remember this film,it was the first film i had watched at the cinema the picture was dark in places i w	positive

LSTM classification on IMDB Dataset

```
import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("IMDB Dataset.csv")

# 2. Show basic information
print(df.head())
print(df["sentiment"].value_counts())

# 3. Clean text
def clean_text(text):
    text = re.sub(r"<.*?>", "", text)
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["review"] = df["review"].apply(clean_text)

# 4. Convert labels to numbers
df["sentiment"] = df["sentiment"].map({"negative": 0, "positive": 1})

# 5. Split text and labels
X_text = df["review"].values
y = df["sentiment"].values
```

LSTM classification on IMDB Dataset—Continued

```
# 6. Train, test split
X_train_text, X_test_text, y_train, y_test = train_test_split(X_text, y, test_size=0.20, random_state=42, stratify=y)

# 7. Tokenization
vocab_size = 10000
max_len = 200
embedding_dim = 100

# The tokenizer keeps the most frequent 10,000 words from the training set.
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>") #<OOV>: out-of-vocabulary token
tokenizer.fit_on_texts(X_train_text)

X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_test_seq = tokenizer.texts_to_sequences(X_test_text)

# 8. Padding and truncation
X_train = pad_sequences(X_train_seq, maxlen=max_len, padding="post", truncating="post")
X_test = pad_sequences(X_test_seq, maxlen=max_len, padding="post", truncating="post")

# 9. Show shapes
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# 10. Build LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_length=max_len, mask_zero=True),
    LSTM(64, activation="tanh"),
    Dropout(0.5),
    Dense(1, activation="sigmoid")
])

# 11. Compile model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

model.summary()
```

Text Preprocessing Before the LSTM

Before text enters the LSTM, it must be converted into numbers.

Raw text $\rightarrow$ Token IDs $\rightarrow$ Word embeddings $\rightarrow$ LSTM

Step 1: Tokenization

“this movie was good” $\rightarrow$ [8, 2, 19, 15]

Each word receives an integer ID from the tokenizer.

Step 2: Padding and truncation

All reviews must have the same length.

[8, 2, 19, 15] $\rightarrow$ [8, 2, 19, 15, 0, 0]

Here, zeros are padding tokens.

Important

The numbers are not word embeddings yet. They are only word IDs. The embedding layer later converts each ID into a dense vector.

What Does post Mean?

In padding and truncation, post means **at the end**.

Post-padding: add zeros after the sequence

$$[8, 2, 19, 15] \rightarrow [8, 2, 19, 15, 0, 0]$$

Post-truncation: cut tokens from the end

$$[4, 9, 12, 7, 5, 11, 20, 3] \rightarrow [4, 9, 12, 7, 5, 11]$$

In our code

```
padding="post", truncating="post"
```

This means short reviews get zeros at the end, and long reviews are cut from the end.

Trainable Embedding

In this example, we used a **trainable embedding layer**:

```
Embedding(input_dim=vocab_size, output_dim=embedding_dim,  
          input_length=max_len, mask_zero=True)
```

This means:

- Each token ID is converted into an embedding vector.
- The embedding vectors start randomly.
- During training, the model updates these vectors.
- The embeddings are learned directly from the IMDB training data.
- The RNN then reads these embeddings token by token.

Key Idea

The embedding layer learns word vectors, and the RNN learns how to use their order for sentiment classification.

- `input_length=max_len` means the sequence length given to the RNN.
- `mask_zero=True` tells the model that token ID 0 is padding, so it should be ignored.

LSTM classification on IMDB Dataset—Continued

```
# 12. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

# 13. Train model
history = model.fit(X_train, y_train, validation_split=0.20, epochs=10, batch_size=32, callbacks=[early_stop], verbose=1)

# 14. Evaluate model
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 15. Predict
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob >= 0.5).astype(int).flatten()

# 16. Extra evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=["Negative", "Positive"]))

# 17. Try one custom review
new_review = ["The movie was not good. It was boring and too long."]

new_review_clean = [clean_text(text) for text in new_review]
new_review_seq = tokenizer.texts_to_sequences(new_review_clean)
new_review_pad = pad_sequences(new_review_seq, maxlen=max_len, padding="post", truncating="post")

prediction = model.predict(new_review_pad)[0][0]

print("Prediction probability:", prediction)

if prediction >= 0.5:
    print("Predicted sentiment: Positive")
else:
    print("Predicted sentiment: Negative")
```

LSTM classification on IMDB Dataset—Result

```

***


| Layer (type)          | Output Shape | Param #     |
|-----------------------|--------------|-------------|
| embedding (Embedding) | ?            | 0 (unbuilt) |
| lstm (LSTM)           | ?            | 0 (unbuilt) |
| dropout (Dropout)     | ?            | 0           |
| dense (Dense)         | ?            | 0 (unbuilt) |



Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
1000/1000 ————— 134s 130ms/step - accuracy: 0.7960 - loss: 0.4504 - val_accuracy: 0.8501 - val_loss: 0.3627
Epoch 2/10
1000/1000 ————— 149s 137ms/step - accuracy: 0.8792 - loss: 0.3112 - val_accuracy: 0.8706 - val_loss: 0.3257
Epoch 3/10
1000/1000 ————— 148s 144ms/step - accuracy: 0.9107 - loss: 0.2346 - val_accuracy: 0.8636 - val_loss: 0.3432
Epoch 4/10
1000/1000 ————— 187s 129ms/step - accuracy: 0.9298 - loss: 0.1876 - val_accuracy: 0.8651 - val_loss: 0.3542
Epoch 5/10
1000/1000 ————— 132s 132ms/step - accuracy: 0.9500 - loss: 0.1418 - val_accuracy: 0.8457 - val_loss: 0.4455
Test Loss: 0.3293164372444153
Test Accuracy: 0.868399977684021
313/313 ————— 11s 33ms/step
Accuracy: 0.8684
Confusion Matrix:
[[4267 733]
 [ 583 4417]]
Classification Report:
              precision    recall  f1-score   support

   Negative       0.88       0.85       0.87       5000
    Positive       0.86       0.88       0.87       5000

   accuracy              0.87       0.87      10000
  macro avg       0.87       0.87       0.87      10000
 weighted avg       0.87       0.87       0.87      10000

1/1 ————— 0s 53ms/step
Prediction probability: 0.10954969
Predicted sentiment: Negative

```

We input token IDs to the model



new_review_pad

#The Embedding layer turns these IDs into word embeddings.

```
... array([[ 2, 17, 13, 21, 49, 9, 13, 355, 3, 100, 204, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0]])
```

**We can also use pre-trained embeddings,
such as GloVe.**

Using Glove for LSTM Classification

```
import pandas as pd
import numpy as np
import re
import gensim.downloader as api

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping

# 1. Load dataset
df = pd.read_csv("IMDB Dataset.csv")

# 2. Show basic information
print(df.head())
print(df["sentiment"].value_counts())

# 3. Clean text
def clean_text(text):
    text = re.sub(r"<.*?>", "", text)
    text = text.lower()
    text = re.sub(r"^[a-zA-Z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["review"] = df["review"].apply(clean_text)

# 4. Convert labels to numbers
df["sentiment"] = df["sentiment"].map({"negative": 0, "positive": 1})

# 5. Split text and labels
X_text = df["review"].values
y = df["sentiment"].values
```

Using GloVe for LSTM Classification—Continued

```
# 6. Train, test split
X_train_text, X_test_text, y_train, y_test = train_test_split(X_text, y, test_size=0.20, random_state=42, stratify=y)

# 7. Tokenization
vocab_size = 10000
max_len = 200
embedding_dim = 100

# The tokenizer keeps the most frequent 10,000 words from the training set.
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>") # <OOV>: out-of-vocabulary token
tokenizer.fit_on_texts(X_train_text)

X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_test_seq = tokenizer.texts_to_sequences(X_test_text)

# 8. Padding and truncation
X_train = pad_sequences(X_train_seq, maxlen=max_len, padding="post", truncating="post")
X_test = pad_sequences(X_test_seq, maxlen=max_len, padding="post", truncating="post")

# 9. Show shapes
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# 10. Load GloVe embeddings using gensim
glove_model = api.load("glove-wiki-gigaword-100")

print("GloVe model loaded")

# 11. Create embedding matrix
word_index = tokenizer.word_index

embedding_matrix = np.zeros((vocab_size, embedding_dim))
```

Using Glove for LSTM Classification—Continued

```
for word, i in word_index.items():
    if i < vocab_size:
        if word in glove_model:
            embedding_matrix[i] = glove_model[word]

print("Embedding matrix shape:", embedding_matrix.shape)

# 12. Build LSTM model
model = Sequential([
    Input(shape=(max_len,)),
    Embedding(input_dim=vocab_size, output_dim=embedding_dim, weights=[embedding_matrix], mask_zero=True, trainable=False),
    LSTM(64, activation="tanh"),
    Dropout(0.5),
    Dense(1, activation="sigmoid")
])

# 13. Compile model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

model.summary()

# 14. Early stopping
early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

# 15. Train model
history = model.fit(X_train, y_train, validation_split=0.20, epochs=10, batch_size=32, callbacks=[early_stop], verbose=1)

# 16. Evaluate model
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 17. Predict
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob >= 0.5).astype(int).flatten()
```

Creating the GloVe Embedding Matrix

```
# 11. Create embedding matrix
word_index = tokenizer.word_index

embedding_matrix = np.zeros((vocab_size, embedding_dim))

for word, i in word_index.items():
    if i < vocab_size: #after fit_on_texts, words are usually indexed by frequency, so the most frequent words receive smaller IDs.
        if word in glove_model:
            embedding_matrix[i] = glove_model[word]
```

This step connects our tokenizer vocabulary to the GloVe vectors.

Tokenizer word ID → Correct GloVe vector

What happens:

- `tokenizer.word_index` gives each word an integer ID.
- `embedding_matrix` starts as a matrix of zeros.
- Each row in this matrix represents one word ID.
- If a word exists in GloVe, we copy its GloVe vector into the correct row.
- If a word is not found in GloVe, its row stays as zeros.

Example:

movie → 17

`embedding_matrix[17]` = GloVe vector for “movie”

Key Idea

The tokenizer gives the word IDs. GloVe gives the vector values. The embedding matrix connects them.

Using Fixed GloVe Embeddings

In our model, we use:

```
trainable=False
```

This means:

- The embedding layer is initialized with GloVe vectors.
- The GloVe vectors are kept fixed.
- The model does not update those embedding vectors during training.
- The LSTM learns from the fixed GloVe representations.

Using Glove for LSTM Classification—Continued

```
# 18. Extra evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=["Negative", "Positive"]))

# 19. Try one custom review
new_review = ["The movie was not good. It was boring and too long."]

new_review_clean = [clean_text(text) for text in new_review]
new_review_seq = tokenizer.texts_to_sequences(new_review_clean)
new_review_pad = pad_sequences(new_review_seq, maxlen=max_len, padding="post", truncating="post")

prediction = model.predict(new_review_pad)[0][0]

print("Prediction probability:", prediction)

if prediction >= 0.5:
    print("Predicted sentiment: Positive")
else:
    print("Predicted sentiment: Negative")
```

Result

```

Test Loss: 0.3157835006713867
Test Accuracy: 0.8644999861717224
313/313 _____ 11s 34ms/step
Accuracy: 0.8645
Confusion Matrix:
[[4416  584]
 [ 771 4229]]
Classification Report:

```

	precision	recall	f1-score	support
Negative	0.85	0.88	0.87	5000
Positive	0.88	0.85	0.86	5000
accuracy			0.86	10000
macro avg	0.87	0.86	0.86	10000
weighted avg	0.87	0.86	0.86	10000

```

1/1 _____ 0s 65ms/step
Prediction probability: 0.16451278
Predicted sentiment: Negative

```

Current Subsection

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)
 - Text Generation
 - Translation

Current Subsubsection

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)
 - Text Generation
 - Translation

Popular Application of RNNs: Text Generation

The most natural application of RNNs is sequence prediction.

In NLP, this usually means predicting the next word from previous words.

previous words $\rightarrow$ RNN/LSTM $\rightarrow$ next word

Example:

“The weather today is” $\rightarrow$ “sunny”

Then the model can repeat the process:

The weather today is sunny

The weather today is sunny and

The weather today is sunny and warm

Key idea:

- RNNs use previous tokens as memory.
- They predict the next token step by step.
- Repeating next-word prediction creates text generation.

Dataset: Tiny Shakespeare

Dataset:

Tiny Shakespeare Dataset

Task: Text generation

Model: RNN, LSTM, or GRU

Input: Sequence of tokens

Output: Next token

Code—LSTM for Text Generation

```
import tensorflow as tf
import numpy as np
import re

# 1. Download Tiny Shakespeare dataset
path_to_file = tf.keras.utils.get_file("shakespeare.txt", "https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.txt")

# 2. Read text
text = open(path_to_file, "rb").read().decode(encoding="utf-8")

print("Length of text:", len(text))
print(text[:500])

# 3. Clean text
def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[a-zA-Z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

text_clean = clean_text(text)

print("Cleaned text sample:")
print(text_clean[:500])

# 4. Split text into word tokens
tokens = text_clean.split()

print("Number of tokens:", len(tokens))
print("First 50 tokens:")
print(tokens[:50])
```

Code—LSTM for Text Generation

```
# 4. Split text into word tokens
tokens = text_clean.split()

print("Number of tokens:", len(tokens))
print("First 50 tokens:")
print(tokens[:50])

# 5. Create vocabulary
vocab = sorted(set(tokens))

print("Vocabulary size:", len(vocab))
print("First 50 vocabulary words:")
print(vocab[:50])

# 6. Create word-to-ID and ID-to-word mappings
word_to_index = {word: i for i, word in enumerate(vocab)}
index_to_word = np.array(vocab)

# 7. Convert all tokens into integer IDs
text_as_int = np.array([word_to_index[word] for word in tokens])

print("First 50 token IDs:")
print(text_as_int[:50])

# 8. Choose sequence length
seq_length = 10

# 9. Create input and target sequences
word_dataset = tf.data.Dataset.from_tensor_slices(text_as_int)

sequences = word_dataset.batch(seq_length + 1, drop_remainder=True)

# Show the first 3 sequences
for seq in sequences.take(3):
    print(seq.numpy())
```

Printing

```
print(word_to_index)
```

```
{'a': 0, 'abandon': 1, 'abase': 2, 'abate': 3, 'abated': 4, 'abbey': 5, 'abbot': 6, 'abed': 7,
```

```
index_to_word
```

```
array(['a', 'abandon', 'abase', ..., 'zenith', 'zodiacs', 'zounds'],  
      dtype='<U15')
```

```
text_as_int
```

```
array([ 3809, 1748,  830, ..., 10034,  512, 10966])
```



```
# Show the first 3 sequences
```

```
for seq in sequences.take(3):
```

```
    print(seq.numpy())
```

```
... [ 3809  1748   830 11050  7608   402  4166  4705  6139  9229   280]  
    [ 9229  9229  3809  1748 11437   476   280  8190  7924 10142  2765]  
    [ 9964 10142  3626   280  8190  8190  3809  1748  3809 11437  5542]
```

The selected sequence length is 10, but each raw chunk contains 11 tokens. The extra token is needed because the target sequence is created by shifting the input sequence one step forward. Therefore, 10 input tokens produce 10 next-token targets. Please note that chunks do not overlap in this setup.

Code—LSTM for Text Generation

```
def split_input_target(chunk):
    input_text = chunk[:-1]
    target_text = chunk[1:]
    return input_text, target_text

dataset = sequences.map(split_input_target)

# 10. Show one training example
for input_example, target_example in dataset.take(1):
    print("Input sequence:")
    print(" ".join(index_to_word[input_example.numpy()]))

    print("\nTarget sequence:")
    print(" ".join(index_to_word[target_example.numpy()]))

# 11. Batch and shuffle data
batch_size = 64
buffer_size = 10000

dataset = dataset.shuffle(buffer_size).batch(batch_size, drop_remainder=True).prefetch(tf.data.AUTOTUNE)
# batch(batch_size, drop_remainder=True): It simply forms batches in order, and if the last batch is incomplete, it removes it.
# shuffle(buffer_size): Keep 10,000 (buffer_size) examples in a buffer, randomly pick from them, then refill the buffer.
# prefetch(tf.data.AUTOTUNE) makes training faster by preparing the next batch while the model is training on the current batch.

# 12. Model parameters
vocab_size = len(vocab)
embedding_dim = 128
lstm_units = 256

# 13. Build LSTM model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=embedding_dim),
    tf.keras.layers.LSTM(lstm_units, return_sequences=True),
    tf.keras.layers.Dense(vocab_size)
])
```

How Does Text Generation Training Work?

Input sequence:

first citizen before we proceed any further hear me speak

Target sequence:

citizen before we proceed any further hear me speak all

Key Idea

The model predicts the next token at every position (not only the final next token).

first	→	citizen
citizen	→	before
before	→	we
we	→	proceed
proceed	→	any
any	→	further
further	→	hear
hear	→	me
me	→	speak
speak	→	all

Why Do We Use `seq_length`?

`seq_length = 10` is the **context window size**.

It means the model learns from chunks of 10 tokens at a time.

Even though the full Shakespeare text is continuous, we cannot feed the whole text at once. So we split it into smaller training sequences.

Example:

Input = first citizen before we proceed any further hear me speak

Target = citizen before we proceed any further hear me speak all

So the model learns:

first $\rightarrow$ citizen, citizen $\rightarrow$ before, ..., speak $\rightarrow$ all

Code—LSTM for Text Generation

```
# 14. Compile model
model.compile(optimizer="adam", loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))

model.summary()

# 15. Train model
epochs = 10

history = model.fit(dataset, epochs=epochs)

# 16. Generate text
def generate_text(model, start_string, num_generate=50, temperature=1.0):
    # Clean the starting text the same way we cleaned the training text
    start_string = clean_text(start_string)

    # Split the starting text into word tokens
    input_tokens = start_string.split()

    # This list will store the integer IDs of the starting words
    input_eval = []

    # Convert each starting word into its integer ID using the training vocabulary
    for word in input_tokens:
        if word in word_to_index:
            input_eval.append(word_to_index[word])

    # If none of the starting words exist in the vocabulary, start with a common word
    if len(input_eval) == 0:
        input_eval = [word_to_index["the"]]

    # Add a batch dimension because the model expects input shape: (batch_size, sequence_length)
    # Example: [12, 45, 8] becomes [[12, 45, 8]]
    input_eval = tf.expand_dims(input_eval, 0)

    # Store the original starting words so we can add generated words after them
    generated_words = input_tokens.copy()
```

Code—LSTM for Text Generation

```
# Generate one word at a time
for i in range(num_generate):
    # Predict the next-word scores for every position in the current input sequence
    predictions = model(input_eval)

    # Keep only the prediction from the last time step
    # Shape changes from (1, sequence_length, vocab_size) to (1, vocab_size)
    # This line takes the model's prediction for only the last word position.
    predictions = predictions[:, -1, :]

    # Control randomness:
    # lower temperature gives safer, more predictable words
    # higher temperature gives more creative, more random words
    predictions = predictions / temperature

    # Randomly sample one word ID from the predicted probability distribution
    predicted_id = tf.random.categorical(predictions, num_samples=1)[-1, 0].numpy()

    # Convert the predicted word ID back into an actual word
    predicted_word = index_to_word[predicted_id]

    # Add the predicted word to the generated text
    generated_words.append(predicted_word)

    # Add the predicted word ID to the input sequence
    # This lets the model use its own generated word to predict the next word
    input_eval = tf.concat([input_eval, tf.expand_dims([predicted_id], 0)], axis=1)

    # Keep only the last seq_length tokens, so the input size stays controlled
    input_eval = input_eval[:, -seq_length:]

# Return the starting words plus all generated words as one text string
return " ".join(generated_words)

# 17. Try text generation
generated_text = generate_text(model, start_string="romeo", num_generate=100, temperature=0.8)

print(generated_text)
```

LSTM Output for Text Generation

In text generation, the model must choose the next token from the vocabulary.

So the final Dense layer has:

`vocab_size`

output neurons.

This means the model produces one score for every possible token:

Output shape = (sequence length, vocab_size)

Example:

`vocab_size = 5000`

The model outputs 5000 scores at each token position.

Key Idea

The LSTM does not directly output a word. It outputs scores for all tokens in the vocabulary, then we select or sample the next token from those scores.

Why Do We Use Only the Last Position During Generation?

During training, the model outputs scores at every token position.

Example input: [first, citizen, ..., me, speak]

The model outputs:

Position	Scores produced
after first	scores for all vocabulary tokens
after citizen	scores for all vocabulary tokens
⋮	⋮
after speak	scores for all vocabulary tokens

During generation, we want the next token after the full current input.

So we only use the last position:

after speak → choose next token

Code

```
predictions = predictions[:, -1, :]
```

Selecting the Next Token

After the model produces scores for all tokens in the vocabulary, those scores are converted into probabilities.

The probabilities sum to 1:

$$P(w_1) + P(w_2) + \dots + P(w_{\text{vocab_size}}) = 1$$

Then one token is sampled based on these probabilities:

```
predicted_id = tf.random.categorical(predictions, num_samples=1)[-1, 0].numpy()
```

Example:

Token	Probability
all	0.45
the	0.25
men	0.20
king	0.10

Key Idea

The model does not always choose the highest-probability token. It samples one token according to the probability distribution.

Temperature in Text Generation

Temperature controls how random the next-token selection is.

Before sampling, we divide the model scores by temperature:

$$\text{adjusted scores} = \frac{\text{model scores}}{\text{temperature}}$$

- **Low temperature** (< 1) makes the model more confident.
- **Temperature = 1** keeps the original model probabilities.
- **High temperature** (> 1) makes the model more random.

Temperature	Effect
0.3	safe, predictable, less creative
1.0	normal randomness
1.5	more creative, more risky

Code—Result

```
Epoch 1/10
296/296 ————— 604s 2s/step - loss: 7.1147
Epoch 2/10
296/296 ————— 597s 2s/step - loss: 6.5288
Epoch 3/10
296/296 ————— 626s 2s/step - loss: 6.3241
Epoch 4/10
296/296 ————— 635s 2s/step - loss: 6.1552
Epoch 5/10
296/296 ————— 610s 2s/step - loss: 5.9996
Epoch 6/10
296/296 ————— 611s 2s/step - loss: 5.8741
Epoch 7/10
296/296 ————— 658s 2s/step - loss: 5.7634
Epoch 8/10
296/296 ————— 626s 2s/step - loss: 5.6625
Epoch 9/10
296/296 ————— 628s 2s/step - loss: 5.5668
Epoch 10/10
296/296 ————— 641s 2s/step - loss: 5.4751
romeo but the devil be either as death and thou art the whole as you buckingham will you were but to them so how she comes so i know your husband
```

You can see this is not a strong model as the generated text does not make sense!

What we created is called a LANGUAGE MODEL!

Reminder: Generative vs. Discriminative Models

Discriminative Models

- Learn to distinguish one class from another.
- Model the **conditional probability**: $P(Y|X)$
- X : input data (e.g., words in an article)
- Y : label or class (e.g., fake or real)
- Focus: how to decide which class a given example belongs to.

Generative Models

- Learn how the data itself is produced.
- Model the **joint probability**: $P(X, Y)$ or the marginal $P(X)$
- X : observed data, Y : underlying label or latent factor
- A joint probability measures how likely two things happen together.
- Focus: how to generate or simulate new data similar to what was observed.

Summary: Discriminative models separate classes. Generative models recreate data.

Language Modeling is Generative

A language model assigns a probability to a sequence of tokens.

$$P(X) = P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

This means the probability of the full sentence is built step by step:

$$P(\text{next token} \mid \text{previous tokens})$$

Key Idea

If a language model is good, it assigns a higher probability to correct or more natural sequences of words.

A language model predicts the next token based on the previous tokens.

Current Subsubsection

1 RNNs in NLP

- RNNs for Classification (Many-to-One)
- RNN for Sequence prediction (Many-to-Many)
 - Text Generation
 - Translation

Popular RNN Application: Machine Translation

A popular many-to-many RNN application is machine translation.

Example task:

English sentence $\rightarrow$ French sentence

Classic model:

Encoder LSTM $\rightarrow$ Context Vector $\rightarrow$ Decoder LSTM

Dataset:

English-French Translation Dataset

Key Idea

The encoder reads the input sentence, and the decoder generates the translated sentence token by token.

Why Translation Needs Encoder-Decoder

Text generation and translation are both sequence tasks, but they are structured differently.

Text generation: one continuous sequence (a decoder-only model)

Input: $[w_1, w_2, w_3]$ $\rightarrow$ Target: $[w_2, w_3, w_4]$

The target is just the input shifted by one token.

Translation: two different sequences (an encoder-decoder model)

English sentence $\rightarrow$ French sentence

The output is not a shifted version of the input.

Encoder LSTM $\rightarrow$ Context Vector $\rightarrow$ Decoder LSTM

Key Idea

The encoder reads the source sentence, and the decoder generates the translated sentence token by token.

Machine Translation

We will discuss this in the next chapter.

Congratulations!

Now you know how RNNs are used in NLP!

